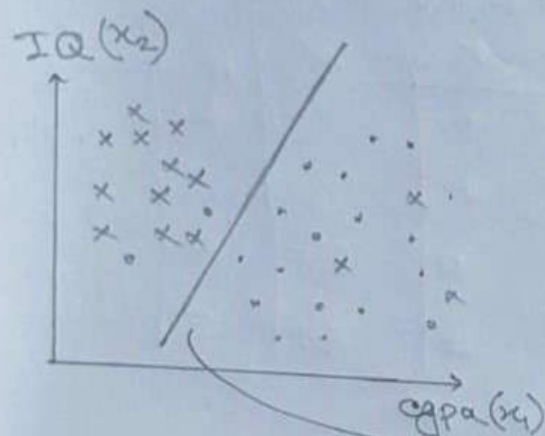
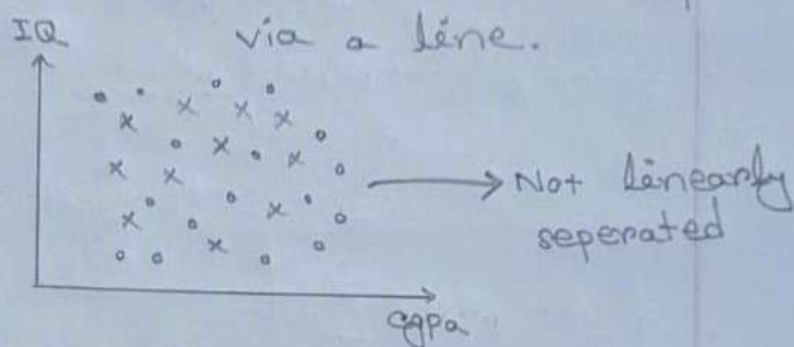


Logistic Regression

It's a binary classification algorithm. output: yes/No
useful when data is linearly separable in 2 classes.



→ If data can be separated via a line.



→ Not linearly separated

#Perceptron Trick:

we have to find the equation of this line.

$AX + BY + C = 0$ → In logistic regression we use general form

$$AX_1 + BX_2 + C = 0$$

For 3 features/dimension

$$AX_1 + BX_2 + CX_3 + d = 0$$

we have to find the value of A B C D so that we can draw the line to separate these 2 class.

Start with random value of coefficients and ^{select a entry randomly}

① ask the random point is the line right?

① If yes → don't converge

② If no → then converge and shift line so that is line falls into right side.

Do this 1000 times or while converging

cost $\uparrow$ rotation ACW
 cost $\downarrow$ rotation CW

How to check a point is on the +ve side / -ve side of a line?

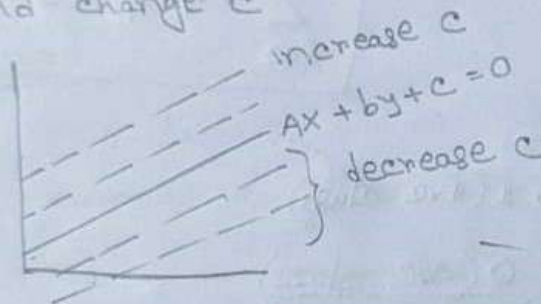
(x_1, y_1) a point and $Ax + by + c = 0$ line

if $Ax_1 + by_1 + c > 0 \rightarrow$ on +ve side

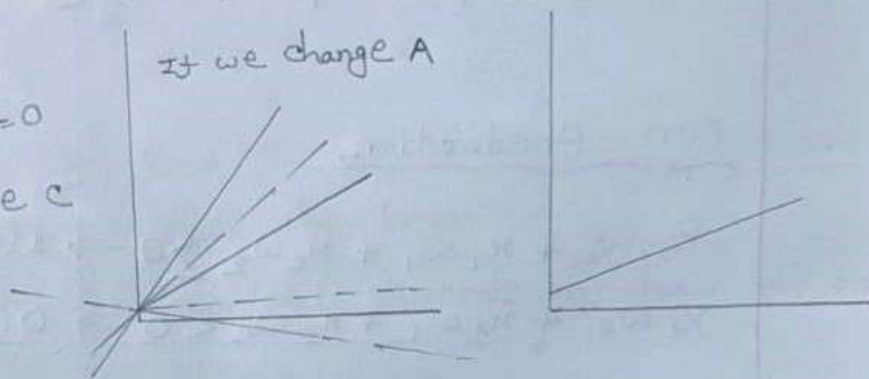
if < 0 -ve side if $Ax_1 + by_1 + c = 0$ then on the line.

$$Ax + by + c = 0$$

if we fix A and B
 and change c



if we change A



initial line
 x region
 x
 (1, 5)

For this point line need to move $\rightarrow$
 so add 1

$$4x + 5y + 3 = 0$$

(1, 5, 1)

$$\begin{array}{r} 1 \quad 5 \quad 1 \\ -1 \quad -3 \quad -2 \\ + \quad + \quad + \end{array}$$

$$\begin{array}{r} 5 \quad 2 \quad 1 \\ 4 \quad 5 \quad 3 \\ -1 \quad -3 \quad -2 \end{array} \rightarrow \text{new line}$$

$$-x - 3y - 2 = 0$$

when wants to move the line towards
 +ve class

when wants to move the line towards
 -ve class

If a -ve point on +ve region:

append 1 to the point coordinate and subtract

If a +ve point on -ve region

append 1 and add to the line cost.

∴ new cost = cost - $\ln x$ (coordinate)

adding new

x_0	x_1	x_2	y (target)
1	7.5	81	0
1	8.5	85	1
1	7	75	0
1	9	105	1

equation

$$w_0 x_0 + w_1 x_1 + w_2 x_2 = 0$$

intercept (c)

$$\sum_{i=0}^2 w_i x_i = 0$$

For Prediction

$$x_0 w_0 + x_1 w_1 + x_2 w_2 \geq 0 \rightarrow 1 \text{ (+ve class)}$$

$$x_0 w_0 + x_1 w_1 + x_2 w_2 < 0 \rightarrow 0 \text{ (-ve class)}$$

epochs = 1000, $\eta = 0.1$

for epoch in range(epochs):

randomly select a row $\rightarrow x_i$

if $x_i \in$ -ve region and $\sum_{i=0}^2 w_i x_i \geq 0$
 update w with $w_{\text{new}} = w_{\text{old}} - \eta * X$

if $x_i \in$ +ve and $\sum_{i=0}^2 w_i x_i < 0$:

$$w_{\text{new}} = w_{\text{old}} + \eta * X$$

Simplified $w_{\text{new}} = w_{\text{old}} + \eta * (y_i - \hat{y}_i) x_i$

+ $\rightarrow$ rotate the line Acw
 - $\rightarrow$ " " " cw

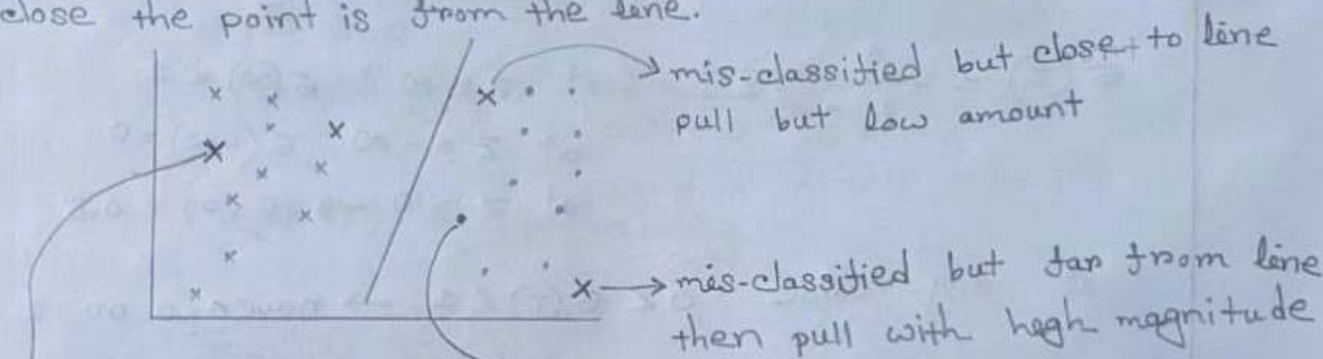
Problem with Perceptron: once we got a line that don't make error on train set perceptron stop improving. So it fails to generalize the problem.

- $\rightarrow$ tends to over-fitting
- $\rightarrow$ linear line mostly closed to a class not perfectly separating.

Perceptron $\rightarrow$ we only update it mis-classified (pull)

Sigmoid Function:

- mis-classified $\rightarrow$ pull the line
- correct classified point $\rightarrow$ push the line
- The amount/magnitude of push/pull depends on how far or close the point is from the line.



correctly classified and far from line then push the line but low magnitude

correct classified and close to line push the line with high magnitude

Perceptron: $w_{\text{new}} = w_{\text{old}} + \eta (\hat{y}_i - y_i) x_i$

if wrong classified $\Rightarrow$ every thing same as previous

if correct $\Rightarrow w_{\text{new}} = w_{\text{old}} \rightarrow$ we have to change it

if correct then $\hat{y}_i - y_i = 0$ we have to prevent this to be zero.

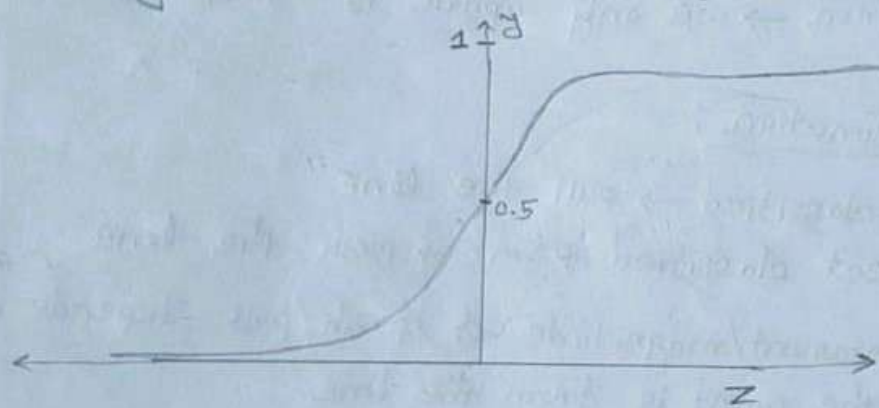
we have to make sure

$$d_i - \hat{d}_i \neq 0$$

we can't do any
thing with this cause it's
real data

we have to change
the prediction method

In perceptron we were using step function so it was only
0 and 1
Instead of using step function now we will use sigmoid function.



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\text{at } z = \infty \quad \sigma(\infty) = 1$$

$$\text{at } z = -\infty \quad \sigma(-\infty) = 0$$

$$\text{at } z = 0 \rightarrow \sigma(0) = 0.5$$

at any value $0 < \sigma(z) < 1 \rightarrow$ never 0 or 1

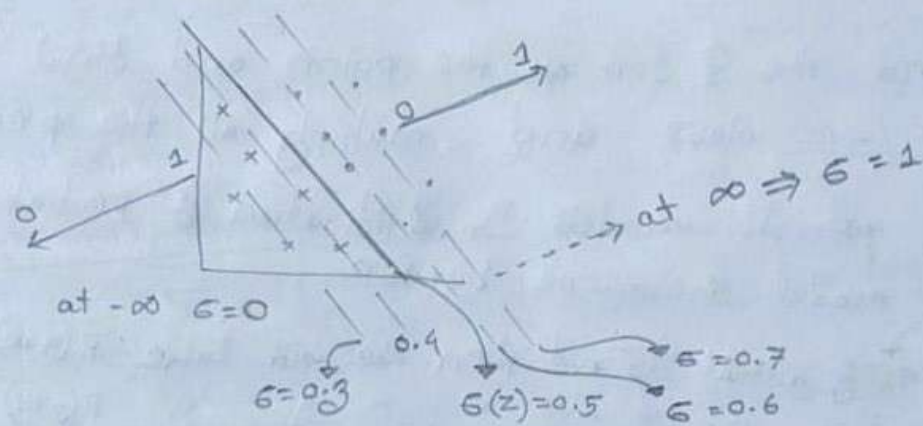
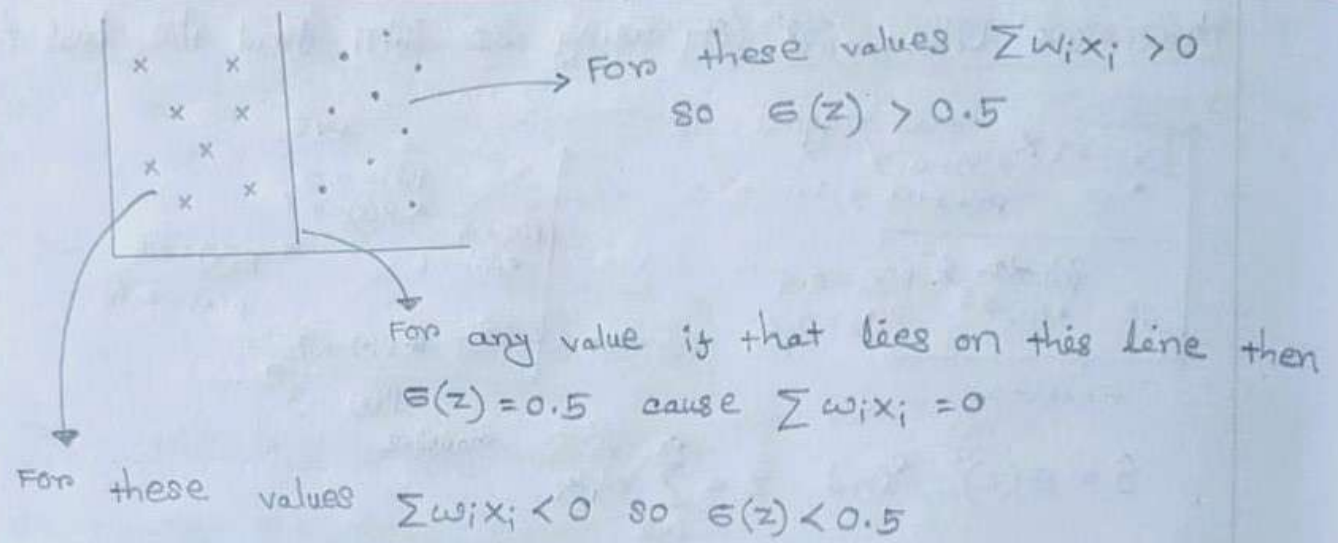
so as before, we will find z . $z = \sum_{i=1}^n w_i x_i$ $n \rightarrow$ columns

then we will pass z to a sigmoid function instead of step function

if z is +ve $\rightarrow \sigma(z) \geq 0.5 \rightarrow$ class-1

if z is -ve $\rightarrow \sigma(z) < 0.5 \rightarrow$ class 0

if z is



we can think $\sigma(z)$ as probability.

Impact of Sigmoid Function

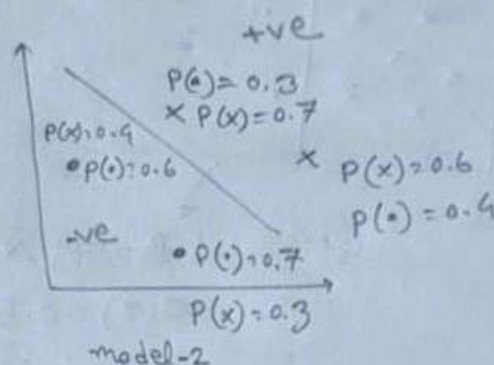
$$w_{\text{new}} = w_{\text{old}} + \eta (\hat{y}_i - \hat{y}_i) x_i$$

$$\hat{y}_i = \sigma(z) \text{ where } z = \sum_{i=1}^n w_i x_i \quad n = \text{no of col.}$$

Here result improves but as we are selecting random points and based on that we are making a line.

There is no guarantee that this is the best line. Cause every time random row will be different and line will be different.

model - 1



$$\hat{g} = \sigma(z) \quad \text{and} \quad z = \sum_{i=1}^n w_i x_i$$

which models multiplication of probability is here that model is more better.

if y actual is +ve then we will take $P(+ve)$
if y " " -ve " " " " $P(-ve)$

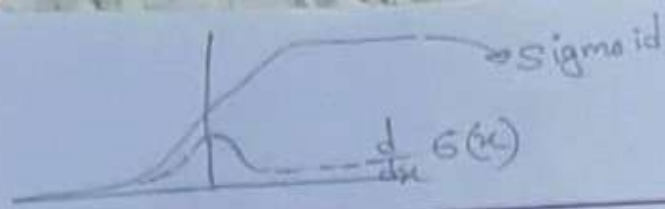
maximum likelihood for model-1 $\Rightarrow 0.4 \times 0.7 \times 0.4 \times 0.8 = 0.089$
 " " " model-2 $\Rightarrow 0.7 \times 0.6 \times 0.6 \times 0.3 = 0.176$

as we are multiplying small numbers and in real world there we be much more rows and this maximum likelihood will be very very small so instead of multiplication we do sum but not direct sum we take the $\log(P)$ and then add.

model-1 $\log(me) = \log(0.4) - \log(0.7) - \log(0.4) - \log(0.8) = -1.0476$

model-2: $\log(0.7) - \log(0.6) - \log(0.6) - \log(0.3) = |-1.12|$ $\rightarrow$ cross

when we sum up all $-\log(m_i)$ we got cross entropy



when we took maximum likelihood we try to maximize it
But in terms of cross entropy we try to minimize it.

for every point calculate

$$-\hat{y}_i \log(\hat{y}_i) - (1 - \hat{y}_i) \log(1 - \hat{y}_i)$$

Then add all values and get the cross entropy.

so finally the loss function

$$L = \sum_{i=1}^m -\hat{y}_i \log(\hat{y}_i) - (1 - \hat{y}_i) \log(1 - \hat{y}_i)$$

$m \rightarrow$ number of
rows

average / mean error

$$L = \frac{-1}{m} \sum_{i=1}^m \hat{y}_i \log(\hat{y}_i) + (1 - \hat{y}_i) \log(1 - \hat{y}_i)$$

log-loss error
on
Binary Cross
Entropy

This function don't have any closed form solution
so we have to minimize this using gradient descent.

Derivative of sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\frac{d}{dx} \sigma(x) = \frac{d}{dx} \left\{ (1 + e^{-x})^{-1} \right\}$$

$$= - (1 + e^{-x})^{-2} \frac{d}{dx} (1 + e^{-x})$$

$$= \frac{-1}{(1 + e^{-x})^2} \times -e^{-x}$$

$$= \frac{e^{-x}}{(1 + e^{-x})^2}$$

$$\rightarrow \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}}$$

$$= \sigma(x) \cdot \frac{1 + e^{-x} - 1}{1 + e^{-x}}$$

$$= \sigma(x) \cdot \left(1 - \frac{1}{1 + e^{-x}} \right)$$

$$= \sigma(x) \{ 1 - \sigma(x) \}$$

$$\therefore \sigma'(x) = \sigma(x) \{ 1 - \sigma(x) \}$$

Step by Step Derivation of average loss function

Step-1 start with 1 sample

$X \rightarrow$ input feature vectors

$y \in \{0, 1\}$ = actual label.

$\theta \rightarrow$ Parameter vector (weights + bias)

$$z = \sum_{i=1}^n \theta_i x_i \Rightarrow z = \theta^T x$$

$$\hat{y} = h_{\theta}(x) = \frac{1}{1 + e^{-z}}$$

we want to define a loss

- Penalizes wrong prediction more
- Encourages high confidence in correct predictions
- works for both class (0 and 1)

Step-2 Define separate Loss for two classes.

$$y \in \{0, 1\}$$

case-1: $y=1$ Loss = $-\log(\hat{y})$ $\rightarrow$ if $\hat{y} \rightarrow 1$ loss = 0 (good)
if $\hat{y} \rightarrow 0$ loss = ∞

case-2: $y=0$ Loss = $-\log(1-\hat{y})$ $\rightarrow$ if $\hat{y} \rightarrow 0$ loss = 0
if $\hat{y} \rightarrow 1$ loss = ∞

Step-3 combine these 2 loss function

$$\text{Loss} = -y_i \log(\hat{y}_i) - (1-y_i) \log(1-\hat{y}_i)$$

$$= -[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

$$\text{if } y=1 \rightarrow (1-y_i) \log(1-\hat{y}_i) = 0$$

$$\text{if } y=0 \rightarrow y_i \log(\hat{y}_i) = 0$$

Step-4 Extend to all training samples

Let's take data set with m samples

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\}$$

$$\text{Loss} = \sum_{i=1}^m \text{Loss}(x^{(i)}, y^{(i)}) = - \sum_{i=1}^m \left[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \right]$$

Gradient Descent:

Rows = m columns = n

					target
1	x_{11}	x_{12}	...	x_{1n}	y_1
1	x_{21}	x_{22}	...	x_{2n}	y_2
...	...	...	...	...	...
...	...	...	...	...	...
1	x_{m1}	x_{m2}	...	x_{mn}	y_n

For n input columns
coefficients are

$$[w_0 \ w_1 \ w_2 \ \dots \ w_n]$$

Total $(n+1)$ weights.

$$\hat{y}_1 = \sigma(w_1 x_{11} + w_2 x_{12} + w_3 x_{13} + \dots + w_n x_{1n} + w_0)$$

$$\hat{y}_2 = \sigma(w_1 x_{21} + w_2 x_{22} + w_3 x_{23} + \dots + w_n x_{2n} + w_0)$$

$$\hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} = \begin{bmatrix} \sigma(w_0 + w_1 x_{11} + w_2 x_{12} + \dots + w_n x_{1n}) \\ \sigma(w_0 + w_1 x_{21} + w_2 x_{22} + \dots + w_n x_{2n}) \\ \vdots \\ \sigma(w_0 + w_1 x_{m1} + w_2 x_{m2} + \dots + w_n x_{mn}) \end{bmatrix}$$

$$\hat{y} = \sigma \begin{pmatrix} \omega_0 + \omega_1 x_{11} + \omega_2 x_{12} + \dots + \omega_n x_{1n} \\ \omega_0 + \omega_1 x_{21} + \omega_2 x_{22} + \dots + \omega_n x_{2n} \\ \vdots \\ \omega_0 + \omega_1 x_{m1} + \omega_2 x_{m2} + \dots + \omega_n x_{mn} \end{pmatrix}$$

$$\hat{y} = \sigma \begin{pmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{pmatrix} \cdot \begin{pmatrix} \omega_0 \\ \omega_1 \\ \vdots \\ \omega_n \end{pmatrix}$$

$$\boxed{\hat{y} = \sigma(Xw)}$$

$$L = -\frac{1}{m} \sum_{i=1}^m \{y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)\}$$

$$= -\frac{1}{m} \left[\sum_{i=1}^m y_i \log(\hat{y}_i) + \sum_{i=1}^m (1-y_i) \log(1-\hat{y}_i) \right]$$

1st term:

$$\begin{aligned} \sum_{i=1}^m y_i \log(\hat{y}_i) &= y_1 \log(\hat{y}_1) + y_2 \log(\hat{y}_2) + \dots + y_m \log(\hat{y}_m) \\ &= [y_1 \ y_2 \ \dots \ y_m] \log \begin{pmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{pmatrix} \\ &= y^T \cdot \log(\hat{y}) \\ &= y^T \cdot \log(\sigma(Xw)) \end{aligned}$$

2nd term: $(1-y)^T \log(1-\sigma(Xw))$

Final Loss function: $L = -\frac{1}{m} \left[y^T \cdot \log(\sigma(Xw)) + (1-y)^T \log(1-\sigma(Xw)) \right]$

$$L = \frac{-1}{m} \left[Y^T \log \hat{Y} + (1-Y)^T \log (1-\hat{Y}) \right] \text{ where}$$

$$\hat{Y} = \sigma(xw)$$

now we have to find this $\frac{\partial L}{\partial w}$ or $\frac{\Delta L}{\Delta w}$

$$\frac{\Delta L}{\Delta w} = \left[\frac{\partial L}{\partial w_0} \quad \frac{\partial L}{\partial w_1} \quad \dots \quad \frac{\partial L}{\partial w_n} \right]$$

and update $w \Rightarrow w := w - \eta * \frac{\Delta L}{\Delta w}$

$$\frac{dL}{dw} = \frac{-1}{m} \frac{d}{dw} \left[Y^T \log(\sigma(xw)) + (1-Y)^T \log(1-\sigma(xw)) \right]$$

$\frac{d}{dx} \ln x = \frac{1}{x}$

1st term: $\frac{d}{dw} (Y^T \log(\sigma(xw))) = \frac{d}{dw} \sum_{i=1}^m Y_i \log(x_i w_i)$

$$= \frac{Y}{\sigma(xw)} \frac{d}{dw} \sigma(xw)$$

$$= \frac{Y}{\hat{Y}} * \sigma(xw) [1 - \sigma(xw)] \frac{d}{dw} (xw)$$

$$= \frac{Y}{\hat{Y}} \sigma(xw) [1 - \sigma(xw)] x$$

$$= \frac{Y}{\hat{Y}} * \hat{Y} (1 - \hat{Y}) x = \boxed{Y (1 - \hat{Y}) x}$$

2nd term:

$$\frac{d}{dw} \left[(1-Y) \log \{1 - \sigma(xw)\} \right]$$

$$= (1-Y) \frac{1}{1 - \sigma(xw)} \frac{\partial}{\partial w} \{1 - \sigma(xw)\}$$

$$= \frac{1-Y}{1-\hat{Y}} \cdot [0 - \sigma(xw) \cdot \{1 - \sigma(xw)\}] \frac{d}{dw} (xw)$$

$$= \frac{1-Y}{(1-\hat{Y})} \cdot -\hat{Y} (1-\hat{Y}) \cdot x = -\hat{Y} (1-Y) x$$

Final gradient of Loss function

$$\frac{\Delta L}{\Delta w} = -\frac{1}{m} \left[y(1-\hat{y})X - \hat{y}(1-y)X \right]$$

$$= -\frac{1}{m} \left[y(1-\hat{y}) - \hat{y}(1-y) \right] X$$

$$= -\frac{1}{m} \left[y - y\hat{y} - \hat{y} + \hat{y}y \right] X$$

$$\boxed{\frac{\Delta L}{\Delta w} = -\frac{1}{m} \left[y - \hat{y} \right] X} \quad \left[\text{note: } y\hat{y} = \hat{y}y \right]$$

$$J(w) = \frac{1}{m} \left[y^T \log(\hat{y}) + (1-y)^T \log(1-\hat{y}) \right]$$

$$= \frac{1}{m} \left[y^T \log(\sigma(xw)) + (1-y)^T \log(1-\sigma(xw)) \right]$$

1st Part:

$$\frac{\partial}{\partial w} \left[y^T \log(\hat{y}) \right]$$

$$= \frac{\partial}{\partial w} \sum_{i=1}^m y_i \log(\hat{y}_i)$$

For a single term: find $\frac{\partial}{\partial w} (y_i \log(\hat{y}_i))$

where $\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}} = \frac{1}{1 + e^{-x_i w}}$
 a sigmoid function

$$\frac{\partial}{\partial w} (y_i \log(\hat{y}_i)) = y_i \cdot \frac{1}{\hat{y}_i} \frac{\partial}{\partial w} (\hat{y}_i)$$

$$= \frac{y_i}{\hat{y}_i} \cdot \hat{y}_i (1 - \hat{y}_i) \frac{\partial}{\partial w} (x_i w)$$

$$= y_i (1 - \hat{y}_i) x_i$$

$$\rightarrow \frac{\partial}{\partial w} \sum_{i=1}^m y_i \log(\hat{y}_i) = \sum_{i=1}^m y_i (1 - \hat{y}_i) x_i \quad \left[\sum a_i x_i = x^T a \right]$$

$$= \boxed{x^T (y - \hat{y})}$$

2nd part: $f(w) = (1-y)^T \log(1-\hat{y})$ where $\hat{y} = \sigma(xw)$

$$\frac{\partial}{\partial w} \left[\sum_{i=1}^m (1-y_i) \log(1-\hat{y}_i) \right]$$

Find derivative for one term

$$\frac{\partial}{\partial w} \left[(1-y_i) \log(1-\hat{y}_i) \right]$$

$$= (1-y_i) \cdot \frac{1}{(1-\hat{y}_i)} \cdot \frac{\partial}{\partial w} (1-\hat{y}_i)$$

$$= (1-y_i) \cdot \frac{-1}{(1-\hat{y}_i)} \cdot \frac{\partial}{\partial w} (\hat{y}_i)$$

$$= (1-y_i) \frac{-1}{(1-\hat{y}_i)} \cdot \hat{y}_i (1-\hat{y}_i) \cdot x_i$$

$$= -(1-y_i) \hat{y}_i x_i$$

$$\rightarrow \frac{\partial}{\partial w} \sum_{i=1}^m \left[(1-y_i) \log(1-\hat{y}_i) \right] = \sum_{i=1}^m -(1-y_i) \hat{y}_i x_i$$

$$= -x^T (1-y) \hat{y}$$

combine Part 1 and 2:

$$\frac{\partial L}{\partial w} = x^T \{ y \cdot (1-\hat{y}) \} - x^T (1-y) \hat{y}$$

$$= x^T \left[y \cdot (1-\hat{y}) - (1-y) \hat{y} \right]$$

$$= x^T \left[y - y\hat{y} - \hat{y} + y\hat{y} \right]$$

$$= x^T (y - \hat{y})$$

$$\therefore \text{final: } \frac{\partial L}{\partial w} = \frac{-1}{m} x^T (y - \hat{y})$$

$$= \frac{1}{m} x^T (\hat{y} - y)$$

Softmax Regression / Multinomial Logistic Regression:

Softmax is the general algorithm and Logistic Regression a sub-type of softmax.

softmax function

$$\sigma(Z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

range: $0 < \sigma < 1$

$\sigma \rightarrow$ softmax

$Z \rightarrow$ input vector

$K \rightarrow$ number of classes

Let's say we have input column
iq cgpa and target $\left\{ \begin{array}{l} \text{yes} \rightarrow 1 \\ \text{no} \rightarrow 2 \\ \text{outout} \rightarrow 3 \end{array} \right\} K=3$

softmax will give the probability of being a class

$$\text{yes} \rightarrow \sigma(Z)_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$\text{no} \rightarrow \sigma(Z)_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

sum of all class probability will be always 1

Dataset

cgpa	iq	target $\in \{0, 1, 2\}$
------	----	--------------------------

apply one hot encoding on target column

cgpa	iq	1	2	3
	1	0	0	0
	0	1	0	0
	0	0	1	0

cgpa | iq | 0
3 weights

cgpa | iq | 1
3 w

cgpa | iq | 2
3 w

make 3 logistic model with 3 dataset.

model-1

$$\omega_0^1 \quad \omega_1^1 \quad \omega_2^1$$

m2

$$\omega_0^2 \quad \omega_1^2 \quad \omega_2^2$$

model-3

$$\omega_0^3 \quad \omega_1^3 \quad \omega_2^3$$

now for prediction take a row and find 3 probability

$$G(\text{Yes}) = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}} \quad G(\text{No}) = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$G(\text{out}) = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

Here basically we make n models where n is the number of classes.

This approach becomes slow if n is high. That's why we don't do this.

Instead of this we change the logistic regression loss function.

Actual loss function of softmax in scikit-learn.

Let, m = number of training examples

K = num. of classes

$$X \in \mathbb{R}^{m \times n}$$

$$W \in \mathbb{R}^{n \times K}$$

$$d \in \{0, 1, \dots, K-1\}$$

$\hat{y}_i^{(k)}$ = Probability that sample i belongs to class k computed using softmax

softmax function $\hat{y}_i^{(k)} = \frac{e^{z_i^k}}{\sum_{j=1}^K e^{z_i^j}}$ where $[z_i = X_i W]$

cross entropy loss (per sample)

$$L^{(i)} = - \sum_{k=1}^K 1(y^{(i)}=k) \log(\hat{y}_i^k)$$

Full Loss over dataset

$$L(w) = - \frac{1}{m} \sum_{i=1}^m \log \left(\frac{e^{z_i^{(y_i)}}}{\sum_{k=1}^K e^{z_i^k}} \right)$$

$$\Rightarrow L(w) = - \frac{1}{m} \sum_{i=1}^m z_i^{(y_i)} - \log \left(\sum_{k=1}^K e^{z_i^k} \right)$$

with regularization

$$J(w) = L(w) + \frac{\lambda}{2} \|w\|^2$$

"Classification Metrics"

Accuracy:

$$\text{accuracy score} = \frac{\text{no. of correct Prediction}}{\text{Total Test size}}$$

⇒ How much accuracy is good?

it depends on the data and Problem statement that we are working.

⇒ Problem with accuracy score:

it just tell how much error model is making but

don't tell which type of error.

↓
This problem solves by confusion matrix

		Prediction			
Actual	0	1	1	0	
	1	True +ve		False -ve	
	1				
	0	False +ve		True -ve	

True +ve: actually true and model is saying true.

" -ve: " false " " is " false

False +ve: " False and " " " true

" -ve: " true " " " false

Accuracy Score from Confusion matrix

$$\left. \begin{array}{l} \text{True +ve} + \text{False -ve} \\ \hline \text{sum of all 4 values} \end{array} \right\} \frac{\text{sum of main diagonal}}{\text{Total of all cell}}$$

Type-1 Error

- False positive
- Healthy person heart diseases

Type-2 Error

- False negative
- heart problem but model saying healthy

When accuracy is misleading
⇒ If dataset is imbalanced.

Precision: What proportion of predicted positives is truly positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

[useful when Type-01 is more dangerous go with high Precision model]

Recall: What proportion of actual positives is correctly classified?

$$\text{Recall} = \frac{TP}{TP + FN}$$

[If Type-2 error is more dangerous go with high Recall model.]

F1 score: [used when we don't know which type error is more dangerous]

There is a trade-off between Precision and Recall.
If we try to increase one another decrease.

→ Harmonic mean of Precision and Recall

$$F1 = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$